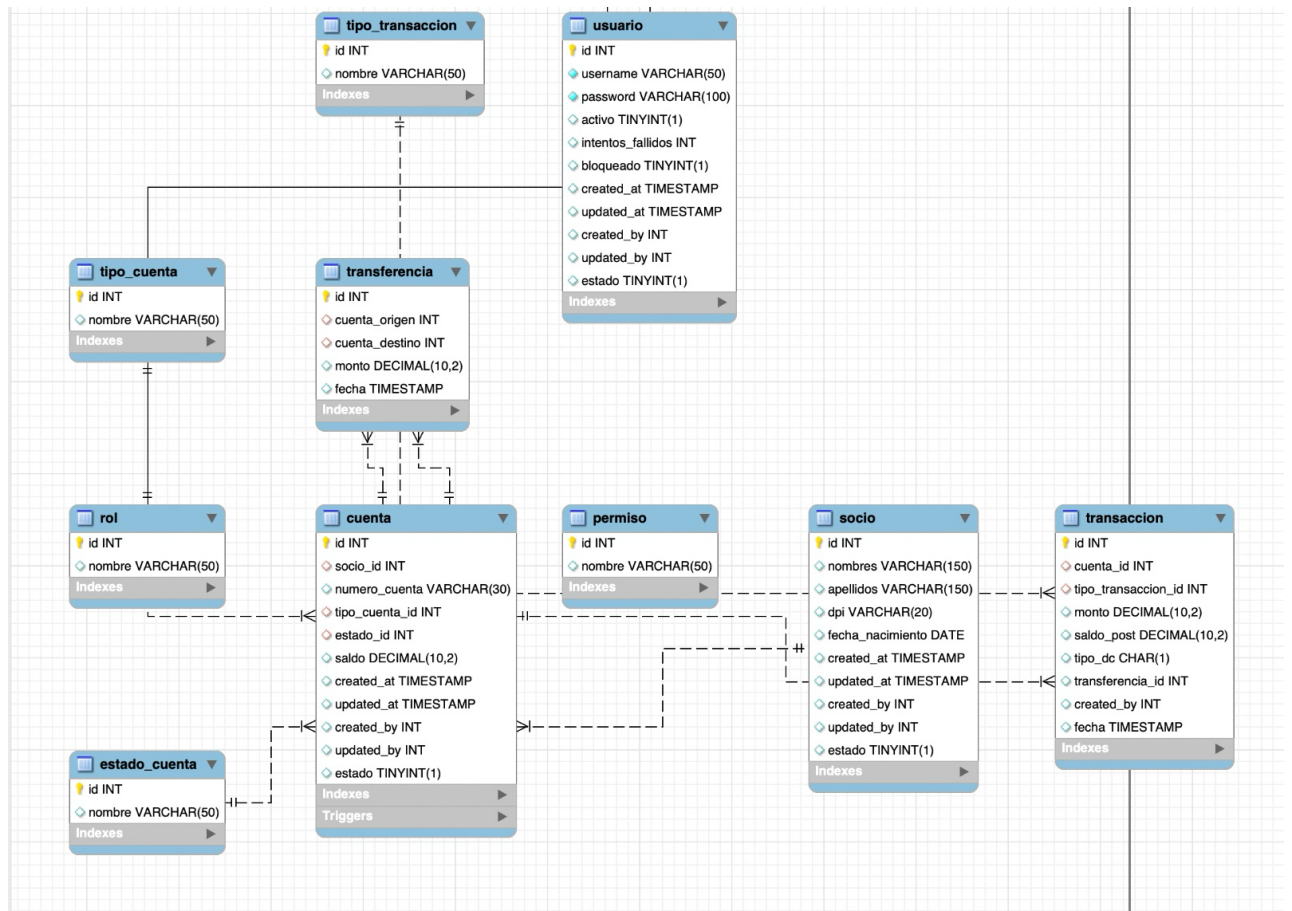


Universidad Mariano Gálvez
Facultad de Ingeniería en Sistemas
Base de datos II
Catedrático: Ing. Jenner Perez
Sección A
Fecha: 31-mayo-2026



Yoselin Karina Ambrosio Avendaño	3490 – 23 - 9197
Francisco Javier Marroquin Martinez	3490 - 23 – 9521
Max de Jesus Ventura Catañeda.	3490 – 23 -9403

Modelo entidad–relación



Datos almacenados

- Clientes

```
43 -- =====
44 -- CLIENTES
45 -- =====
46
47 • CREATE TABLE socio (
48     id INT AUTO_INCREMENT PRIMARY KEY,
49     nombres VARCHAR(150),
50     apellidos VARCHAR(150),
51     dpi VARCHAR(20) UNIQUE,
52     fecha_nacimiento DATE,
53     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
54     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
55     created_by INT,
56     updated_by INT,
57     estado BOOLEAN DEFAULT TRUE
58 );
59
60 -- =====
61
-- APERTURA CUENTA
CREATE PROCEDURE sp_apertura_cuenta(
    IN p_socio INT,
    IN p_tipo INT,
    IN p_estado INT,
    IN p_saldo DECIMAL(10,2),
    IN p_usuario INT
)
BEGIN
181     IF p_saldo < 0 THEN
182         SIGNAL SQLSTATE '45000'
183         SET MESSAGE_TEXT = 'Saldo inválido';
184     END IF;
185
186     INSERT INTO cuenta(
187         socio_id, numero_cuenta, tipo_cuenta_id,
188         estado_id, saldo, created_by
189     )
190     VALUES(
191         p_socio,
192         CONCAT('CTA-', SUBSTRING(UUID(),1,8)), -- UUID recortado
193         p_tipo,
194         p_estado,
195         p_saldo,
196         p_usuario
197     );
198 END;
199 //
200
```

- Depositar

```
201  -- DEPOSITAR
202  • CREATE PROCEDURE sp_depositar(
203      IN p_cuenta INT,
204      IN p_monto DECIMAL(10,2),
205      IN p_usuario INT
206  )
207  BEGIN
208      DECLARE saldo_actual DECIMAL(10,2);
209
210      START TRANSACTION;
211
212      SELECT saldo INTO saldo_actual
213      FROM cuenta
214      WHERE id = p_cuenta
215      FOR UPDATE;
216
217      IF saldo_actual IS NULL THEN
218          ROLLBACK;
219          SIGNAL SQLSTATE '45000'
220          SET MESSAGE_TEXT = 'Cuenta no existe';
221      END IF;
222
223      UPDATE cuenta
224      SET saldo = saldo + p_monto,
225          updated_by = p_usuario
226      WHERE id = p_cuenta;
227
228      INSERT INTO transaccion
229      (cuenta_id, tipo_transaccion_id, monto, saldo_post, tipo_dc, created_by)
230      VALUES (p_cuenta, 1, p_monto, saldo_actual + p_monto, 'C', p_usuario);
231
232      COMMIT;
233  END;
234  //
```

- Retirar

```
236  -- RETIRAR
237  • CREATE PROCEDURE sp_retirar(
238      IN p_cuenta INT,
239      IN p_monto DECIMAL(10,2),
240      IN p_usuario INT
241  )
242  BEGIN
243      DECLARE saldo_actual DECIMAL(10,2);
244
245      START TRANSACTION;
246
247      SELECT saldo INTO saldo_actual
248      FROM cuenta
249      WHERE id = p_cuenta
250      FOR UPDATE;
251
252      IF saldo_actual < p_monto THEN
253          ROLLBACK;
254          SIGNAL SQLSTATE '45000'
255          SET MESSAGE_TEXT = 'Fondos insuficientes';
256      END IF;
257  END;
```



```

257
258     UPDATE cuenta
259     SET saldo = saldo - p_monto
260     WHERE id = p_cuenta;

261
262     INSERT INTO transaccion
263     VALUES (NULL, p_cuenta, 2, p_monto, saldo_actual - p_monto, 'D', NULL, p_usuario, NOW());
264
265     COMMIT;
266 END;
267 //

```

- Transferir

```

269 -- TRANSFERIR
270 • CREATE PROCEDURE sp_transferir(
271     IN p_origen INT,
272     IN p_destino INT,
273     IN p_monto DECIMAL(10,2),
274     IN p_usuario INT
275 )
276 BEGIN
277     DECLARE saldo_origen DECIMAL(10,2);
278     DECLARE v_id INT;
279
280     START TRANSACTION;
281
282     SELECT saldo INTO saldo_origen
283     FROM cuenta
284     WHERE id = p_origen
285     FOR UPDATE;
286
287     SELECT saldo FROM cuenta
288     WHERE id = p_destino
289     FOR UPDATE;
290
291     IF saldo_origen < p_monto THEN
292         ROLLBACK;
293         SIGNAL SQLSTATE '45000'
294         SET MESSAGE_TEXT = 'Fondos insuficientes';
295     END IF;
296
297     UPDATE cuenta SET saldo = saldo - p_monto WHERE id = p_origen;
298     UPDATE cuenta SET saldo = saldo + p_monto WHERE id = p_destino;
299
300     INSERT INTO transferencia(cuenta_origen, cuenta_destino, monto)
301     VALUES(p_origen, p_destino, p_monto);
302
303     SET v_id = LAST_INSERT_ID();
304
305     INSERT INTO transaccion
306     VALUES (NULL, p_origen, 2, p_monto, saldo_origen - p_monto, 'D', v_id, p_usuario, NOW());
307
308     INSERT INTO transaccion
309     VALUES (NULL, p_destino, 1, p_monto,
310         (SELECT saldo FROM cuenta WHERE id = p_destino),

```

```

311     'C', v_id, p_usuario, NOW());
312
313     COMMIT;
314 END;
315 //
316
317 DELIMITER ;
318

```

Vistas

- Cuenta_socios

```

1 • CREATE VIEW vw_cuentas_socios AS
2 SELECT
3     c.id AS cuenta_id,
4     c.numero_cuenta,
5     CONCAT(s.nombres, ' ', s.apellidos) AS socio,
6     tc.nombre AS tipo_cuenta,
7     ec.nombre AS estado_cuenta,
8     c.saldo
9 FROM cuenta c
10 INNER JOIN socio s ON c.socio_id = s.id
11 INNER JOIN tipo_cuenta tc ON c.tipo_cuenta_id = tc.id
12 INNER JOIN estado_cuenta ec ON c.estado_id = ec.id;

```

- Transacciones

```

1 • CREATE VIEW vw_transacciones AS
2 SELECT
3     t.id,
4     c.numero_cuenta,
5     tt.nombre AS tipo_transaccion,
6     t.monto,
7     t.saldo_post,
8     t.fecha
9 FROM transaccion t
10 INNER JOIN cuenta c ON t.cuenta_id = c.id
11 INNER JOIN tipo_transaccion tt ON t.tipo_transaccion_id = tt.id;
12

```

Triggers

- Asigna automáticamente el año a los registros históricos.

```
344  -- =====
345  -- TRIGGER PARA LLENAR EL AÑO
346  -- =====
347  DELIMITER //
348  • CREATE TRIGGER trg_set_anio_hist
349    BEFORE INSERT ON transaccion_hist
350    FOR EACH ROW
351  BEGIN
352    SET NEW.anio = YEAR(NEW.fecha);
353  END;
354  //
355  DELIMITER ;
```

- Copia automáticamente las transacciones a la tabla histórica particionada.

```
357  -- =====
358  -- TRIGGER PARA COPIAR DATOS DESDE TRANSACCION
359  -- =====
360  DELIMITER //
361  • CREATE TRIGGER trg_copy_to_hist
362    AFTER INSERT ON transaccion
363    FOR EACH ROW
364  BEGIN
365    INSERT INTO transaccion_hist (
366      cuenta_id, tipo_transaccion_id, monto, saldo_post,
367      tipo_dc, transferencia_id, created_by, fecha, anio
368    )
369    VALUES (
370      NEW.cuenta_id, NEW.tipo_transaccion_id, NEW.monto,
371      NEW.saldo_post, NEW.tipo_dc, NEW.transferencia_id,
372      NEW.created_by, NEW.fecha, YEAR(NEW.fecha)
373    );
374  END;
375  //
376  DELIMITER ;
```

- Evita saldos negativos en cuentas

```
133  -- =====
134  -- TRIGGERS
135  -- =====
136  DELIMITER //
137
138  • CREATE TRIGGER trg_no_saldo_negativo
139    BEFORE UPDATE ON cuenta
140    FOR EACH ROW
141  BEGIN
142    IF NEW.saldo < 0 THEN
143    END;
144  END;
145  //
```

Evidencia de control de concurrencia.

Está dentro de los procedimientos almacenados

- sp_depositar

```
210      START TRANSACTION;
211
212      SELECT saldo INTO saldo_actual
213      FROM cuenta
214      WHERE id = p_cuenta
215      FOR UPDATE;
216
217      IF saldo_actual IS NULL THEN
218          ROLLBACK;
219          SIGNAL SQLSTATE '45000'
220          SET MESSAGE_TEXT = 'Cuenta no existe';
221      END IF;
222
223      UPDATE cuenta
224      SET saldo = saldo + p_monto,
225          updated_by = p_usuario
226      WHERE id = p_cuenta;
227
228      INSERT INTO transaccion
229      (cuenta_id, tipo_transaccion_id, monto, saldo_post, tipo_dc, created_by)
230      VALUES (p_cuenta, 1, p_monto, saldo_actual + p_monto, 'C', p_usuario);
231
232      COMMIT;
233  END;
234  //
235
236  RETURN
```

- sp_retirar

```
245      START TRANSACTION;
246
247      SELECT saldo INTO saldo_actual
248      FROM cuenta
249      WHERE id = p_cuenta
250      FOR UPDATE;
251
252      IF saldo_actual < p_monto THEN
253          ROLLBACK;
254          SIGNAL SQLSTATE '45000'
255          SET MESSAGE_TEXT = 'Fondos insuficientes';
256      END IF;
257
258      UPDATE cuenta
259      SET saldo = saldo - p_monto
260      WHERE id = p_cuenta;
261
262      INSERT INTO transaccion
263      VALUES (NULL, p_cuenta, 2, p_monto, saldo_actual - p_monto, 'D', NULL, p_usuario, NOW());
264
265      COMMIT;
266  END;
267  //
```


- sp_transferir.

```
269 -- TRANSFERIR
270 • CREATE PROCEDURE sp_transferir(
271     IN p_origen INT,
272     IN p_destino INT,
273     IN p_monto DECIMAL(10,2),
274     IN p_usuario INT
275 )
276 BEGIN
277     DECLARE saldo_origen DECIMAL(10,2);
278     DECLARE v_id INT;
279
280     START TRANSACTION;
281
282     SELECT saldo INTO saldo_origen
283     FROM cuenta
284     WHERE id = p_origen
285     FOR UPDATE;
286
287     SELECT saldo FROM cuenta
288     WHERE id = p_destino
289     FOR UPDATE;
290
291     IF saldo_origen < p_monto THEN
292         ROLLBACK;
293         SIGNAL SQLSTATE '45000'
294         SET MESSAGE_TEXT = 'Fondos insuficientes';
295     END IF;
296
297     UPDATE cuenta SET saldo = saldo - p_monto WHERE id = p_origen;
298     UPDATE cuenta SET saldo = saldo + p_monto WHERE id = p_destino;
299
300     INSERT INTO transferencia(cuenta_origen, cuenta_destino, monto)
301     VALUES(p_origen, p_destino, p_monto);
302
303     SET v_id = LAST_INSERT_ID();
304
305     INSERT INTO transaccion
306     VALUES (NULL, p_origen, 2, p_monto, saldo_origen - p_monto, 'D', v_id, p_usuario, NOW());
307
308     INSERT INTO transaccion
309     VALUES (NULL, p_destino, 1, p_monto,
310         (SELECT saldo FROM cuenta WHERE id = p_destino),
311         'C', v_id, p_usuario, NOW());
312
313     COMMIT;
314
315 END;
316 //
```


Implementación de particionamiento.

```
320 • -- =====
321 -- TABLA HISTÓRICA PARTICIONADA
322 -- =====
323
324 ○ CREATE TABLE transaccion_hist (
325     id INT AUTO_INCREMENT,
326     cuenta_id INT,
327     tipo_transaccion_id INT,
328     monto DECIMAL(10,2),
329     saldo_post DECIMAL(10,2),
330     tipo_dc CHAR(1),
331     transferencia_id INT NULL,
332     created_by INT,
333     fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
334     anio INT,
335     PRIMARY KEY (id, anio) -- Clave primaria compuesta para permitir particionamiento
336 )
337 ○ PARTITION BY RANGE (anio) (
338     PARTITION p2024 VALUES LESS THAN (2025),
339     PARTITION p2025 VALUES LESS THAN (2026),
340     PARTITION p2026 VALUES LESS THAN (2027),
341     PARTITION pmax VALUES LESS THAN MAXVALUE
342 );
343
```

Evidencia de optimización de consultas.

- Evidencia de optimización de consultas.

```
93 • CREATE INDEX idx_cuenta_socio ON cuenta(socio_id);
94
```

- Índice sobre transacción

```
118
119 • CREATE INDEX idx_transaccion_cuenta ON transaccion(cuenta_id);
120
```

Código fuente del backend.

Desarrollado para la base de datos Cooperativa incluye la creación de todas las tablas necesarias para la gestión de usuarios, socios, cuentas y transacciones financieras. Además, incorpora restricciones de integridad referencial mediante claves primarias y foráneas, procedimientos almacenados para la automatización de operaciones bancarias (creación de socios, apertura de cuentas, depósitos, retiros y transferencias), triggers para validar reglas de negocio y mantener un historial de transacciones, así como índices para optimizar el rendimiento de las consultas. También se implementó control de concurrencia mediante transacciones y bloqueos de registros (FOR UPDATE), y particionamiento de la tabla histórica de transacciones para mejorar la administración y el rendimiento de grandes volúmenes de datos. Este script constituye la base de datos principal del sistema y soporta las operaciones fundamentales de la cooperativa.

Datos de seguridad

- Roles
- Permisos
- Usuario administrador
- Asignación de roles

```
7  -- =====
8  -- DATOS DE SEGURIDAD
9  -- =====
10 • INSERT INTO rol (nombre) VALUES ('Administrador'), ('Cajero'), ('Socio');
11 • INSERT INTO permiso (nombre) VALUES ('CrearCuenta'), ('Depositar'), ('Retirar'), ('Transferir');
12
13 -- Usuario con contraseña encriptada
14 • INSERT INTO usuario (username, password, created_by)
15   VALUES ('karina', SHA2('MiPasswordSeguro', 512), 1);
16
17 -- Asignar rol al usuario
18 • INSERT INTO usuario_rol (usuario_id, rol_id) VALUES (1, 1);
19
```

Datos de clientes

Utiliza el procedimiento almacenado para registrar socios

```
20  -- =====
21  -- DATOS DE CLIENTES
22  -- =====
23 • CALL sp_crear_socio('Juan', 'Pérez', '9876543210101', '1990-05-12', 1);
24 • CALL sp_crear_socio('María', 'López', '1234567890102', '1992-07-20', 1);
25
--
```

Datos de cuenta

Crea tipos de cuenta, estados y abre cuentas para los socios.

```
26  -- =====
27  -- DATOS DE CUENTAS
28  -- =====
29  • INSERT INTO tipo_cuenta (nombre) VALUES ('Ahorro'), ('Corriente');
30  • INSERT INTO estado_cuenta (nombre) VALUES ('Activa'), ('Inactiva');
31
32  -- Apertura de cuentas
33  • CALL sp_apertura_cuenta(1, 1, 1, 1000.00, 1); -- Cuenta de Juan
34  • CALL sp_apertura_cuenta(2, 2, 1, 500.00, 1); -- Cuenta de María
35
```

Tipos de transacciones

Carga el catálogo de tipos de transacciones

```
--
36  -- =====
37  -- TIPOS DE TRANSACCIÓN
38  -- =====
39  • INSERT INTO tipo_transaccion (nombre) VALUES ('Depósito'), ('Retiro'), ('Transferencia');
40
```

Pruebas de operaciones

Prueba que los procedimientos almacenados funcionen correctamente.

```
--
41  -- =====
42  -- PRUEBAS DE OPERACIONES
43  -- =====
44  -- Depositar en cuenta de Juan
45  • CALL sp_depositar(1, 200.00, 1);
46
47  -- Retirar de cuenta de María
48  • CALL sp_retirar(2, 100.00, 1);
49
50  -- Transferir de Juan a María
51  • CALL sp_transferir(1, 2, 150.00, 1);
52
```

Verificación

Muestra los resultados para comprobar que los datos fueron creados y que los triggers y procedimientos funcionaron

```
53  -- =====
54  -- VERIFICACIÓN
55  -- =====
56  • SELECT * FROM cuenta;
57  • SELECT * FROM usuario;
58  • SELECT * FROM transaccion;
59  • SELECT * FROM transferencia;
60  • SELECT * FROM transaccion_hist;
```


La captura muestra probando procedimientos almacenados en MariaDB

Master:

```
mariadb-master [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

max@max-VirtualBox: ~
-----+
| cuenta | CREATE TABLE `cuenta` (
| `id` int(11) NOT NULL AUTO INCREMENT,
| `socio_id` int(11) DEFAULT NULL,
| `numero_cuenta` varchar(50) DEFAULT NULL,
| `tipo_cuenta_id` int(11) DEFAULT NULL,
| `estado_id` int(11) DEFAULT NULL,
| `saldo` decimal(10,2) DEFAULT NULL CHECK (`saldo` >= 0),
| `created_at` timestamp NULL DEFAULT current_timestamp(),
| `updated_at` timestamp NULL DEFAULT current_timestamp() ON UPDATE current_timestamp(),
| `created_by` int(11) DEFAULT NULL,
| `updated_by` int(11) DEFAULT NULL,
| `estado` tinyint(1) DEFAULT 1,
| PRIMARY KEY (`id`),
| UNIQUE KEY `numero_cuenta` (`numero_cuenta`),
| KEY `tipo_cuenta_id` (`tipo_cuenta_id`),
| KEY `estado_id` (`estado_id`),
| KEY `idx_cuenta_socio` (`socio_id`),
| CONSTRAINT `cuenta_ibfk_1` FOREIGN KEY (`socio_id`) REFERENCES `socio` (`id`) ON UPDATE CASCADE,
| CONSTRAINT `cuenta_ibfk_2` FOREIGN KEY (`tipo_cuenta_id`) REFERENCES `tipo_cuenta` (`id`) ON UPDATE CASCADE,
| CONSTRAINT `cuenta_ibfk_3` FOREIGN KEY (`estado_id`) REFERENCES `estado_cuenta` (`id`) ON UPDATE CASCADE
| ) ENGINE=InnoDB AUTO_INCREMENT=3 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci |
+-----+

1 row in set (0.004 sec)

MariaDB [cooperativa]> call sp_crear_socio('Jenner', 'Perez', 1614388222222, '19770520',2);
'> ^C
MariaDB [cooperativa]> call sp_crear_socio('Jenner', 'Perez', 1614388222222, '1990-05-20',2);
'> ^C
MariaDB [cooperativa]> call sp_crear_socio('Jenner', 'Perez', 1614388222222, '1990-05-20',2);
Query OK, 1 row affected (0.028 sec)

MariaDB [cooperativa]> call sp_apertura_cuenta(3,1,1,2.00,1);
Query OK, 1 row affected (0.018 sec)

MariaDB [cooperativa]> call sp_restirar(3,200.00,1);
ERROR 1305 (42000): PROCEDURE cooperativa.sp_restirar does not exist
MariaDB [cooperativa]> call sp_retirar(3,200.00,1);
ERROR 1644 (45000): Fondos insuficientes
MariaDB [cooperativa]> 
```

Slave:

```

max@max-VirtualBox: ~
$ mariadb-slave [Oracle] [Oracle] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

1 | NULL | 1 |
+-----+-----+
1 row in set (0.001 sec)

MariaDB [cooperativa]> SELECT * FROM socio
>
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | nombres | apellidos | dpi | fecha_nacimiento | created_at | updated_at | created_by | updated_by | estado |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | Juan | Pérez | 9876543210101 | 1990-05-12 | 2026-05-31 14:01:05 | 2026-05-31 14:01:05 | 1 | NULL | 1 |
| 2 | María | López | 1234567890102 | 1992-07-20 | 2026-05-31 14:01:05 | 2026-05-31 14:01:05 | 1 | NULL | 1 |
| 3 | Jenner | Pérez | 1614388222222 | 1990-05-20 | 2026-05-31 14:17:17 | 2026-05-31 14:17:17 | 2 | NULL | 1 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.002 sec)

MariaDB [cooperativa]> select * from tipo_cuenta;
+-----+-----+
| id | nombre |
+-----+-----+
| 1 | Ahorro |
| 2 | Corriente |
+-----+-----+
2 rows in set (0.000 sec)

MariaDB [cooperativa]> select * from usuario;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | username | password | created_by | updated_by | estado | activo | intentos_fallidos | blo |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | karina | bd8632c77ac761c22a11b6e1dd4955e4406d483f44e5ec23f60ee0b746ed7fe468c307a2c3a086e85440aef9a64988624071349e1da82e4c54f06692a8a24ef2a | 1 | NULL | 1 | 1 | 0 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.001 sec)

MariaDB [cooperativa]> select * from cuenta;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | socio_id | numero_cuenta | tipo_cuenta_id | estado_id | saldo | created_at | updated_at | created_by | updated_by | estado |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | 1 | CTA-72c7f5e5 | 1 | 1 | 1050.00 | 2026-05-31 14:01:05 | 2026-05-31 14:01:05 | 1 | 1 | 1 |
| 2 | 2 | CTA-72c88435 | 2 | 1 | 550.00 | 2026-05-31 14:01:05 | 2026-05-31 14:01:05 | 1 | NULL | 1 |
| 3 | 3 | CTA-654f2597 | 1 | 1 | 2.00 | 2026-05-31 14:22:11 | 2026-05-31 14:22:11 | 1 | NULL | 1 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.000 sec)

MariaDB [cooperativa]>

```

Master:

[illegible]

```
row in set (0.004 sec)

MariaDB [cooperativa]> call sp_crear_socio('Jenner', 'Perez', 1614388222222, '19770520',2);
^C
MariaDB [cooperativa]> call sp_crear_socio('Jenner', 'Perez', 1614388222222, '1990-05-20',2);
^C
MariaDB [cooperativa]> call sp_crear_socio('Jenner', 'Perez', 1614388222222, '1990-05-20',2);
Query OK, 1 row affected (0.028 sec)

MariaDB [cooperativa]> call sp_apertura_cuenta(3,1,1,2,00,1);
Query OK, 1 row affected (0.018 sec)

MariaDB [cooperativa]> call sp_retirar(3,200,00,1);
ERROR 1305 (42000): PROCEDURE cooperativa.sp_retirar does not exist
MariaDB [cooperativa]> call sp_retirar(3,200,00,1);
ERROR 1644 (45000): Fondos insuficientes
MariaDB [cooperativa]> SHOW PROCEDURE STATUS WHERE Db = 'cooperativa';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Db      | Name      | Type      | Definer      | Modified     | Created     | Security type | Comment      | character_set_client | collation_connection | Database Collation |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| cooperativa | sp_apertura_cuenta | PROCEDURE | root@localhost | 2026-05-31 13:58:05 | 2026-05-31 13:58:05 | DEFINER      |              | utf8mb3              | utf8mb3_general_ci  | utf8mb4_general_ci |
| cooperativa | sp_crear_socio      | PROCEDURE | root@localhost | 2026-05-31 13:58:05 | 2026-05-31 13:58:05 | DEFINER      |              | utf8mb3              | utf8mb3_general_ci  | utf8mb4_general_ci |
| cooperativa | sp_depositar         | PROCEDURE | root@localhost | 2026-05-31 13:58:05 | 2026-05-31 13:58:05 | DEFINER      |              | utf8mb3              | utf8mb3_general_ci  | utf8mb4_general_ci |
| cooperativa | sp_retirar          | PROCEDURE | root@localhost | 2026-05-31 13:58:05 | 2026-05-31 13:58:05 | DEFINER      |              | utf8mb3              | utf8mb3_general_ci  | utf8mb4_general_ci |
| cooperativa | sp_transferir        | PROCEDURE | root@localhost | 2026-05-31 13:58:05 | 2026-05-31 13:58:05 | DEFINER      |              | utf8mb3              | utf8mb3_general_ci  | utf8mb4_general_ci |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
5 rows in set (0.018 sec)

MariaDB [cooperativa]> 
```

```
5 rows in set (0.018 sec)
```

```
MariaDB [cooperativa]> SHOW TABLES;
```

Tables_in_cooperativa
cuenta
estado_cuenta
permiso
rol
socio
tipo_cuenta
tipo_transaccion
transaccion
transaccion_hist
transferencia
usuario
usuario_rol

```
12 rows in set (0.002 sec)
```

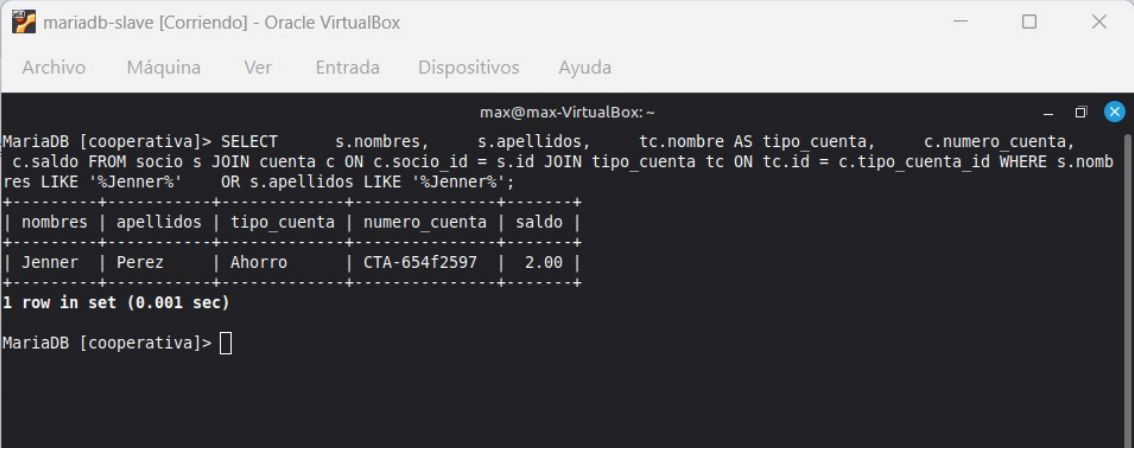
```
MariaDB [cooperativa]> 
```

```
mysql> [cooperative]- SHOW TRIGGERS FROM cooperative;
```

Trigger	Event	Table	Statement	Definer	character_set_client	collation_connection	Database Collation	Timing	Created	sql_mode
trg_no_saldo_negativo	UPDATE	cuenta	BEGIN IF NEW.saldo < 0 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'No se permite saldo negativo'; END IF; END	root@localhost	utf8mb3	utf8mb3_general_ci	utf8mb3_general_ci	BEFORE	2026-05-31 13:58:05.25	STRICT_TRANS_TABLES,ERROR_FOR_DIVISION_BY_ZERO,NO_AUTO_CREATE_USER,NO_ENGINE_SUBSTITUTION
trg_copyp_his	INSERT	transaccion_hist	trg copy to hist INSERT transaccion_hist BEGIN INSERT INTO transaccion_hist (cuenta_id, tipo_transaccion_id, monto, saldo_post, tipo_dc, transferencia_id, created_by, fecha, anio) VALUES (NEW.cuenta_id, NEW.tipo_transaccion_id, NEW.monto, NEW.saldo_post, NEW.tipo_dc, NEW.transferencia_id, NEW.created_by, NEW.fecha, YEAR(NEW.fecha)); END	root@localhost	utf8mb3	utf8mb3_general_ci	utf8mb3_general_ci	AFTER	2026-05-31 13:58:05.53	STRICT_TRANS_TABLES,ERROR_FOR_DIVISION_BY_ZERO,NO_AUTO_CREATE_USER,NO_ENGINE_SUBSTITUTION
trg_set_anio_hist	INSERT	transaccion_hist	BEGIN SET NEW.anio = YEAR(NEW.fecha); END	root@localhost	utf8mb3	utf8mb3_general_ci	utf8mb3_general_ci	BEFORE	2026-05-31 13:58:05.51	STRICT_TRANS_TABLES,ERROR_FOR_DIVISION_BY_ZERO,NO_AUTO_CREATE_USER,NO_ENGINE_SUBSTITUTION

3 rows in set (0.011 sec)

Consulta de cuenta:



The screenshot shows a terminal window titled "mariadb-slave [Corriendo] - Oracle VirtualBox". The terminal displays a SQL query and its results. The query selects names, surnames, account type, account number, and balance for a specific person. The results are formatted as a table with one row.

```
mariadb-slave [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

max@max-VirtualBox: ~
MariaDB [cooperativa]> SELECT      s.nombres,      s.apellidos,      tc.nombre AS tipo_cuenta,      c.numero_cuenta,
c.saldo FROM socio s JOIN cuenta c ON c.socio_id = s.id JOIN tipo_cuenta tc ON tc.id = c.tipo_cuenta_id WHERE s.nomb
res LIKE '%Jenner%' OR s.apellidos LIKE '%Jenner%';
+-----+-----+-----+-----+-----+
| nombres | apellidos | tipo_cuenta | numero_cuenta | saldo |
+-----+-----+-----+-----+-----+
| Jenner  | Perez     | Ahorro      | CTA-654f2597  | 2.00  |
+-----+-----+-----+-----+-----+
1 row in set (0.001 sec)

MariaDB [cooperativa]> 
```

Repositorio Github

<https://github.com/francisco3490239521/proyecto-final-base-de-datos-II.git>